

FIȘA 4 - Subprograme, fișiere text și aplicații modulare

Date generale	Disciplina: Informatică Clasa: a IX-a Durata recomandată: 50 minute Tip activitate: fișă de lucru - teorie, exemplu, problemă
---------------	--

Tipul 1. Definirea și apelarea unui subprogram

Partea teoretică. Un subprogram este o secvență de instrucțiuni cu nume propriu, care poate primi date prin parametri și poate întoarce un rezultat prin return. Subprogramele fac programul mai clar și mai ușor de testat.

Exemplu rezolvat. Definiți o funcție care calculează aria unui dreptunghi.

Funcția primește lungimea și lățimea, calculează produsul și întoarce rezultatul.

```
def aria_dreptunghi(lungime, latime):  
    return lungime * latime  
  
L = float(input("Lungime = "))  
l = float(input("Latime = "))  
print("Aria este", aria_dreptunghi(L, l))
```

Problemă propusă. Scrieți o funcție care primește baza și înălțimea unui triunghi și returnează aria acestuia.

Indicație / soluție pe scurt: Formula este $\text{baza} \times \text{înălțimea} / 2$.

Tipul 2. Funcții predefinite pentru calcule și colecții

Partea teoretică. Python oferă funcții predefinite utile: abs() pentru valoare absolută, round() pentru rotunjire, int() și float() pentru conversii, len(), min(), max(), sum() pentru colecții.

Exemplu rezolvat. Se citește o listă de temperaturi. Determinați minimul, maximumul și media.

Folosim min(), max(), sum() și len(), evitând scrierea manuală a unor parcurgeri.

```
temperaturi = [float(x) for x in input("Temperaturi: ").split()]  
print("Minim:", min(temperaturi))  
print("Maxim:", max(temperaturi))  
print("Media:", round(sum(temperaturi) / len(temperaturi), 2))
```

Problemă propusă. Se citește o listă de vânzări lunare. Afișați totalul, vânzarea minimă, vânzarea maximă și media rotunjită la două zecimale.

Indicație / soluție pe scurt: Folosiți sum(), min(), max(), len() și round().

Tipul 3. Citirea și scrierea în fișiere text

Partea teoretică. Lucrul cu fișiere presupune deschiderea fișierului, citirea sau scrierea datelor și închiderea lui. În Python, este recomandată instrucțiunea with, deoarece închide automat fișierul.

Exemplu rezolvat. Din fișierul numere.txt se citesc numere întregi. Scrieți în rezultat.txt suma lor.

Citim tot conținutul fișierului, îl împărțim în valori, convertim la int, calculăm suma și scriem rezultatul.

```
with open("numere.txt", "r") as f:
    valori = [int(x) for x in f.read().split()]

s = sum(valori)

with open("rezultat.txt", "w") as g:
    g.write(str(s))
```

Problemă propusă. Din fișierul note.txt se citesc note întregi. Scrieți în promovati.txt doar notele mai mari sau egale cu 5.

Indicație / soluție pe scurt: Citiți lista de note, filtrați valorile ≥ 5 și scrieți-le separate prin spațiu.

Tipul 4. Proiectare modulară: citire, prelucrare, afișare

Partea teoretică. O soluție modulară împarte problema în părți mai mici: citirea datelor, prelucrarea datelor și afișarea rezultatului. Fiecare funcție trebuie să aibă un rol clar.

Exemplu rezolvat. Realizați un program modular care calculează raportul unei liste de note: minim, maxim, medie.

Definim funcții separate pentru citire, calcul și afișare. Astfel, programul este mai ușor de urmărit și modificat.

```
def citeste_note():
    return [int(x) for x in input("Note: ").split()]

def raport(note):
    return min(note), max(note), sum(note) / len(note)

def afiseaza(minim, maxim, media):
    print("Minim:", minim)
    print("Maxim:", maxim)
    print("Media:", round(media, 2))

note = citeste_note()
mi, ma, med = raport(note)
afiseaza(mi, ma, med)
```

Problemă propusă. Proiectați modular o aplicație care primește o listă de cheltuieli și afișează totalul, media și cheltuielile peste medie.

Indicație / soluție pe scurt: Folosiți funcții pentru citire, calculul mediei și filtrarea valorilor peste medie.

Autoevaluare

- Pot defini o funcție cu parametri și rezultat returnat.
- Pot utiliza funcții predefinite pentru colecții.
- Pot lucra cu fișiere text și pot organiza o aplicație în module.

Temă opțională

Alegeți una dintre problemele propuse și scrieți programul complet în Python. Testați programul pentru cel puțin trei seturi de date: un caz obișnuit, un caz limită și un caz cu valori repetate sau extreme.

LevelUP Edu